

NỀN TẢNG AI TẠO SINH
(IT5410 – Foundation of Generative AI)

MỘT SỐ VẤN ĐỀ CỦA LÝ THUYẾT HỌC MÁY VÀ HỌC SÂU

Thân Quang Khoát
Trường CNTT&TT, ĐHBKHN

2024

Nội dung

- Mở đầu
- **Một số vấn đề của Học sâu**
- Một số kiến trúc mạng nơron
- Mô hình sinh sâu
- Đánh giá chất lượng
- Học tăng cường

Learning theory

Basic concepts

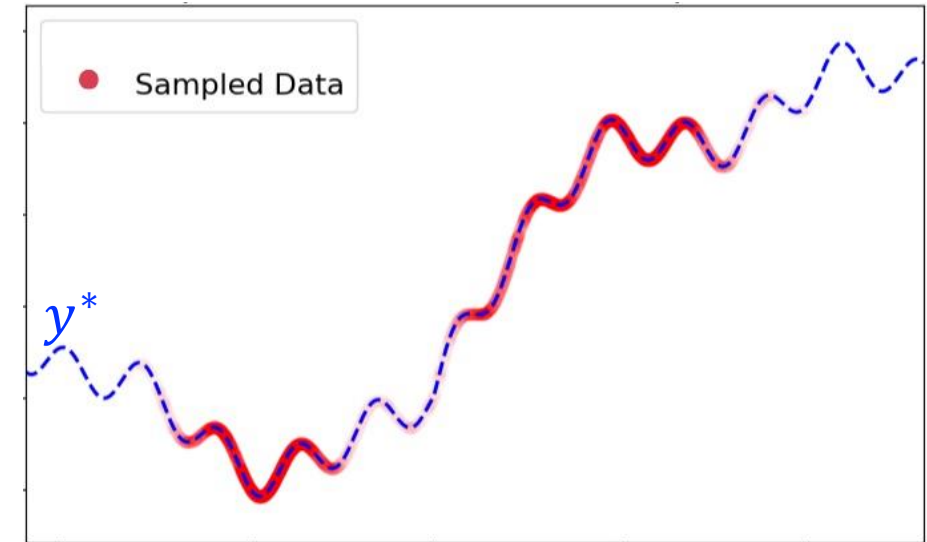
Lý thuyết học tập
Khái niệm cơ bản

The supervised learning problem

- There is an **unknown** (measurable) function (Có một hàm gán nhãn nào đó)

$$y^*: \mathcal{X} \mapsto \mathcal{Y}$$

- It maps each input $x \in \mathcal{X}$ to a label (output) $y \in \mathcal{Y}$
- Sometimes known as the *labeling function*
- Spaces: *input space* $\mathcal{X}$, *output space* $\mathcal{Y}$
- We can collect a dataset $\mathbf{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$
 - $y_i = y^*(\mathbf{x}_i)$ for any $i \in \{1, \dots, m\}$
 - $\mathbf{x}_i$ is an observation about input x (một quan sát về x)
- We need to learn y^* from $\mathbf{D}$

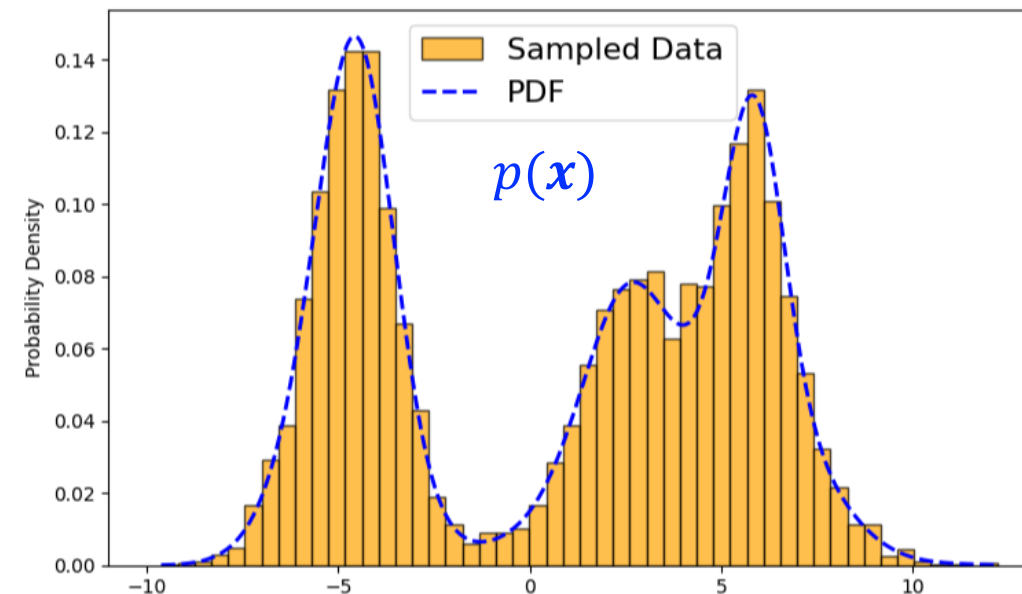
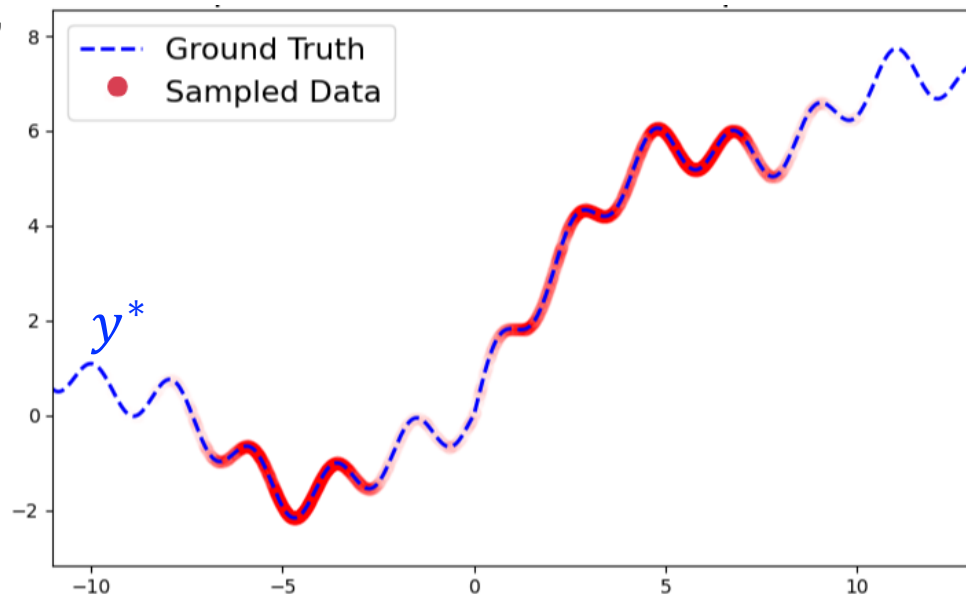


The sampling distribution

- Dataset $\mathbf{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$
 - Each $\mathbf{x}_i$ is collected/generated according to a distribution P , i.e., $\mathbf{x}_i$ is a sample from distribution P
 - P is often **unknown** in practice.
- Those samples are often assumed to be i.i.d.
(Independent and identically distributed – Độc lập và được sinh ra từ một phân bố)

The
data-generation
model

- Easy for analysis, each example may contain maximal information



The sampling distribution

- Dataset $\mathbf{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$ 2 n s khó gii quy
- **Hardness** of the learning problem: The two unknown
 - y^* in the space of all measurable functions
 - $P(\mathbf{x})$ in the space of all probability distributions
- In practice, we often find a function h to **approximate** y^*
 - h is often called **predictor** or *prediction rule* i ng vòng: tìm hàm h xp x y^*



Impossible

Learning algorithm

- **Learning algorithm (Learner):** an algorithm that can return one predictor h , based on a given training set $\mathbf{D}$

$$\mathcal{A}: \mathbf{D} \mapsto h$$

- Receive a training set $\mathbf{D}$,
- Can access the input space $\mathcal{X}$ and label space $\mathcal{Y}$
- Return one predictor $h: \mathcal{X} \rightarrow \mathcal{Y}$
- What space does a learning algorithm map from?
- E.g., K-nearest neighbors, ordinary least square
- The learner **may not know** any thing about the true y^* and $P(\mathbf{x})$

(Thuật toán học thường không biết gì thêm về hàm y^* và phân bố $P(\mathbf{x})$)

Hypothesis space

- Usually, we can choose a restricted set $\mathcal{H}$ of functions

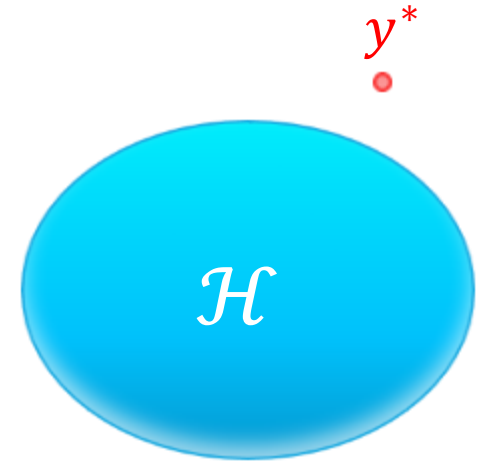
- A learner must find one $h \in \mathcal{H}$

- Our hope: $h(\mathbf{x}) \approx y^*(\mathbf{x}), \quad \forall \mathbf{x} \in \mathcal{X}$

- Hypothesis space (*model space*):

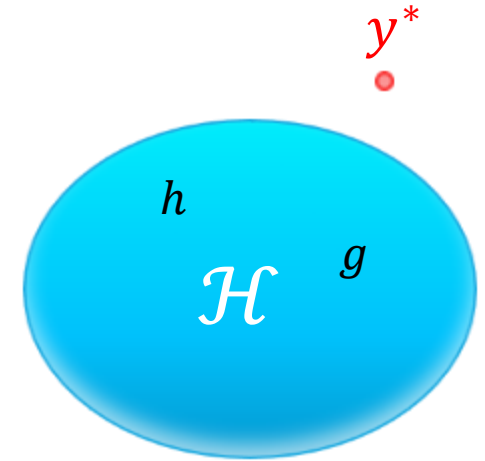
a set $\mathcal{H}$ of functions, providing candidates h for a learning algorithm

- Each candidate depends on input features: $h: \mathcal{X} \rightarrow \mathcal{Y}$
 - Represents prior knowledge about a task
 - Represents our **inductive bias** or *preference*
- Each h is often called a “**model**” or “**predictor**”
- E.g., Linear model: $\mathcal{H} = \{h(\mathbf{x}, \mathbf{w}) = w_0 + w_1x_1 + \dots + w_nx_n: w_0, w_1, \dots, w_n \in \mathbb{R}\}$



Loss function

- How to see the goodness of each $h \in \mathcal{H}$?
 - How well does h predict each input $\mathbf{x}$?
 - How to compare two predictors h and g ?
- **Loss (cost) function:** $f: \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$
 - $f(y, \hat{y})$: the cost/loss of prediction $\hat{y}$ about y
 - Tells the quality of each prediction
- **Examples:**
 - 0-1 loss: $f(y, \hat{y}) = \mathbf{1}_{y \neq \hat{y}}$
 - Square loss: $f(y, \hat{y}) = (y - \hat{y})^2$
 - Absolute loss: $f(y, \hat{y}) = |y - \hat{y}|$
 - Hinge loss: $f(y, \hat{y}) = \max(0, 1 - y\hat{y})$



For vectors:

- Square (ℓ_2) loss: $f(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$
- Absolute (ℓ_1) loss: $f(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|_1$

Loss or Risk

- When h makes a prediction about $\mathbf{x}$, its loss is $f(y^*(\mathbf{x}), h(\mathbf{x}))$
- **Empirical loss (risk)**: the loss of a predictor h on the training set $\mathbf{D}$

$$F(\mathbf{D}, h) = \frac{1}{M} \sum_{i=1}^M f(y_i, h(\mathbf{x}_i))$$

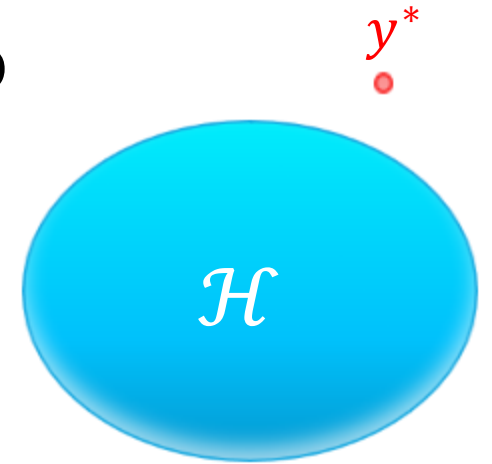
- Sometimes known as **Empirical risk**
- **Expected loss (risk)**: the loss over the whole space

$$F(P, h) = \mathbb{E}_{\mathbf{x} \sim P}[f(y^*(\mathbf{x}), h(\mathbf{x}))]$$

- Sometimes known as **Expected risk**
- For binary classification, where $\mathcal{Y} = \{0, 1\}$, and absolute loss f :

$$\Pr(y^*(\mathbf{x}) \neq h(\mathbf{x})) = F(P, h)$$

- For $\mathcal{Y} = \{-1, 1\}$ and absolute loss f : $\Pr(y^*(\mathbf{x}) \neq h(\mathbf{x})) = \frac{1}{2}F(P, h)$



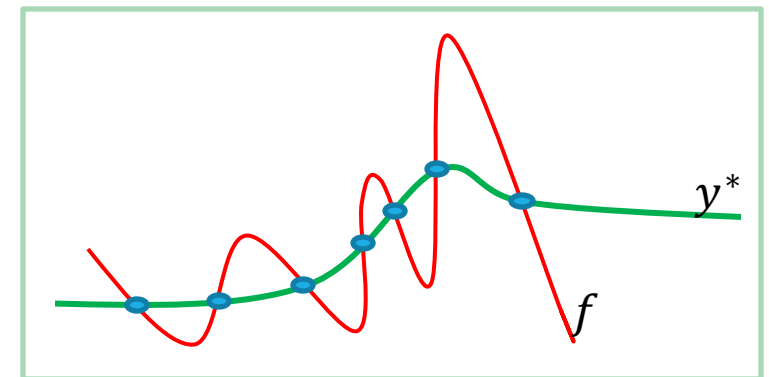
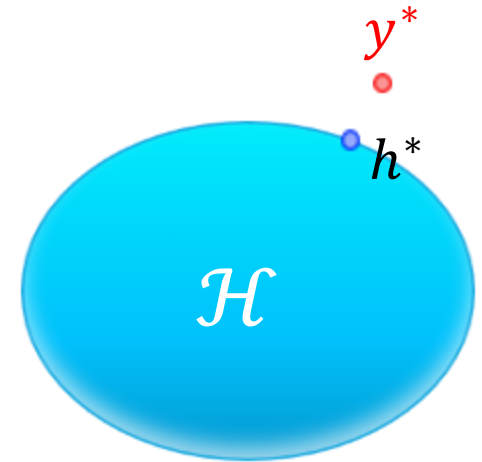
Learning goal

- Learner's task: find a good $h \in \mathcal{H}$, based on a training set $\mathbf{D}$
- *Learning goal*: find one $h \in \mathcal{H}$ with *small expected loss*
 - It should **generalize** well on future data instances
 - Small training loss/error is not enough
- Ultimately, we want to find the best one in $\mathcal{H}$

$$h^* = \arg \min_{h \in \mathcal{H}} F(P, h)$$

- **Learning $\neq$ Fitting**

- Fitting focuses on minimizing the training loss $F(\mathbf{D}, h)$



Empirical Risk Minimization (ERM)

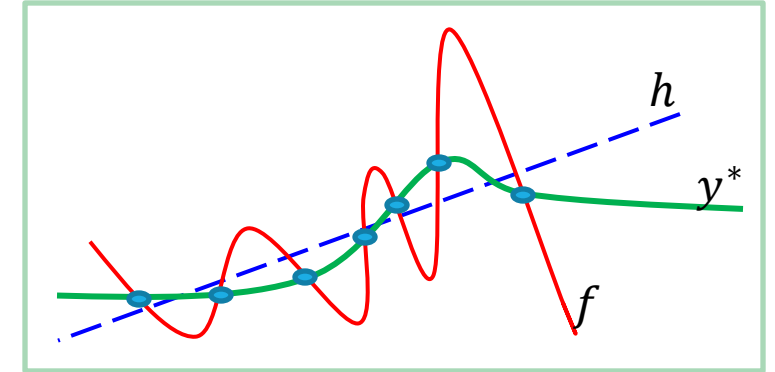
- In practice, a learner cannot access $F(P, h)$
- It should rely on $F(\mathbf{D}, h)$
 - Can be computed from a training set $\mathbf{D}$

- ERM will find

$$h_{erm} = \arg \min_{h \in \mathcal{H}} F(\mathbf{D}, h)$$

- Focuses on fitting (minimizing the training error)
 - However, h_{erm} may get overfitting
- ERM is good in the cases of large training set, due to

$$F(\mathbf{D}, h) \xrightarrow{m \rightarrow \infty} F(P, h)$$



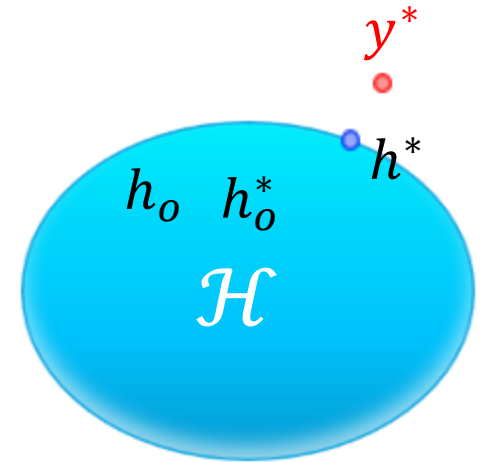
- Regularization-based algorithms:

$$h_{reg} = \arg \min_{h \in \mathcal{H}} F(\mathbf{D}, h) + \lambda \cdot g(h)$$

- $g(h)$ is a penalty on h , λ is a regularization constant
 - This penalty helps us avoid too complex candidates and hence reduce overfitting

Error of a model

- After training, the learning algorithm will return $h_o \in \mathcal{H}$
- How well does it work with future data? $\Rightarrow$ **Generalization!**
- Maybe: $h_o \neq h_o^*$ and $h_o \neq h^*$
 - $h_o^* = \arg \min_{h \in \mathcal{H}} F(\mathbf{D}, h)$ is the *minimizer of the empirical loss*
 - $h^* = \arg \min_{h \in \mathcal{H}} F(P, h)$ is the *best member* of family $\mathcal{H}$



Note:

$$F(P, h_o) - F(P, y^*) = F(P, h_o) - F(P, h^*) + F(P, h^*) - F(P, y^*)$$

- **Estimation error:** $F(P, h_o) - F(P, h^*)$
 - How good is the training algorithm?
- **Approximation error:** $F(P, h^*) - F(P, y^*)$
 - Capacity (representational power) of family $\mathcal{H}$

Error decomposition

- Estimation error

$$\begin{aligned} & F(P, h_o) - F(P, h^*) \\ &= F(P, h_o) - F(\mathbf{D}, h_o) + F(\mathbf{D}, h_o) - F(\mathbf{D}, h_o^*) + F(\mathbf{D}, h_o^*) - F(P, h^*) \end{aligned}$$

- It can be decomposed into two types of error: **Optimization** + **Generalization**

- Note that

$$\begin{aligned} |F(P, h_o) - F(\mathbf{D}, h_o)| &\leq \sup_{h \in \mathcal{H}} |F(P, h) - F(\mathbf{D}, h)| \\ |F(\mathbf{D}, h_o^*) - F(P, h^*)| &= \left| \min_{h \in \mathcal{H}} F(\mathbf{D}, h) - \min_{h \in \mathcal{H}} F(P, h) \right| \leq \sup_{h \in \mathcal{H}} |F(P, h) - F(\mathbf{D}, h)| \end{aligned}$$

- Therefore

$$|F(P, h_o) - F(P, h^*)| \leq \underbrace{|F(\mathbf{D}, h_o) - F(\mathbf{D}, h_o^*)|}_{\text{(Optimization error)}} + 2 \underbrace{\sup_{h \in \mathcal{H}} |F(P, h) - F(\mathbf{D}, h)|}_{\text{(Generalization error)}}$$

$$|F(P, h_o) - F(P, h^*)| \leq |F(\mathbf{D}, h_o) - F(\mathbf{D}, h_o^*)| + 2 \sup_{h \in \mathcal{H}} |F(P, h) - F(\mathbf{D}, h)|$$

- Estimation error can be decomposed into two types of error

- **Optimization error:** $F(\mathbf{D}, h_o) - F(\mathbf{D}, h_o^*)$

- How close to optimality is h_o ?

- h_o may not be the global minimum of the training loss

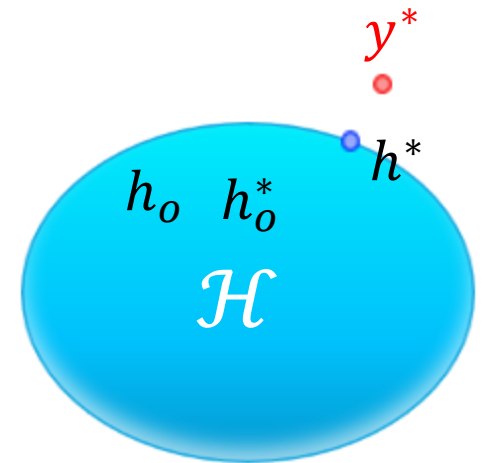
- **Generalization error:** $F(P, h_o) - F(\mathbf{D}, h_o)$

- How far is the training loss from expected loss?

- Smaller should be better

- In summary:

thông thng không tính c chính xác tng loi li, c lng nó.
chia thành 3 loi li c bn tìm ra ngun gc li



$$Error(h_o) := \text{Optimization error} + \text{Generalization error} + \text{Approximation error}$$

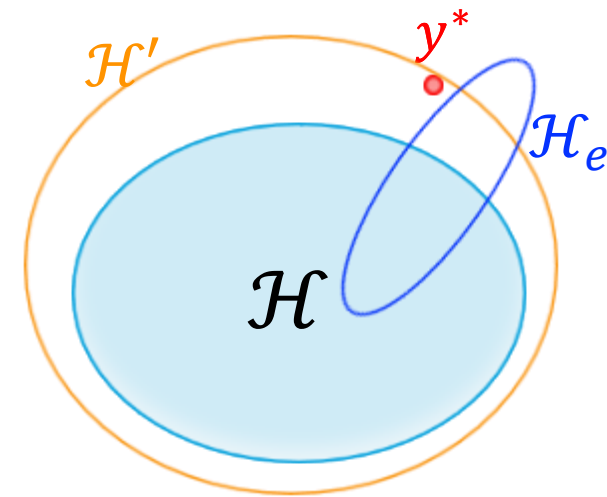
Errors by different factors

■ Hypothesis space $\mathcal{H}$

- A bigger space ($\mathcal{H}'$), the (probably) smaller approximation error → **Why?**
- More complex members, the (probably) smaller approximation error → **larger capacity**
- An *effective* space ($\mathcal{H}_e$) is enough → not too big/complex
- **Realizable** case: $F(P, h^*) = 0$

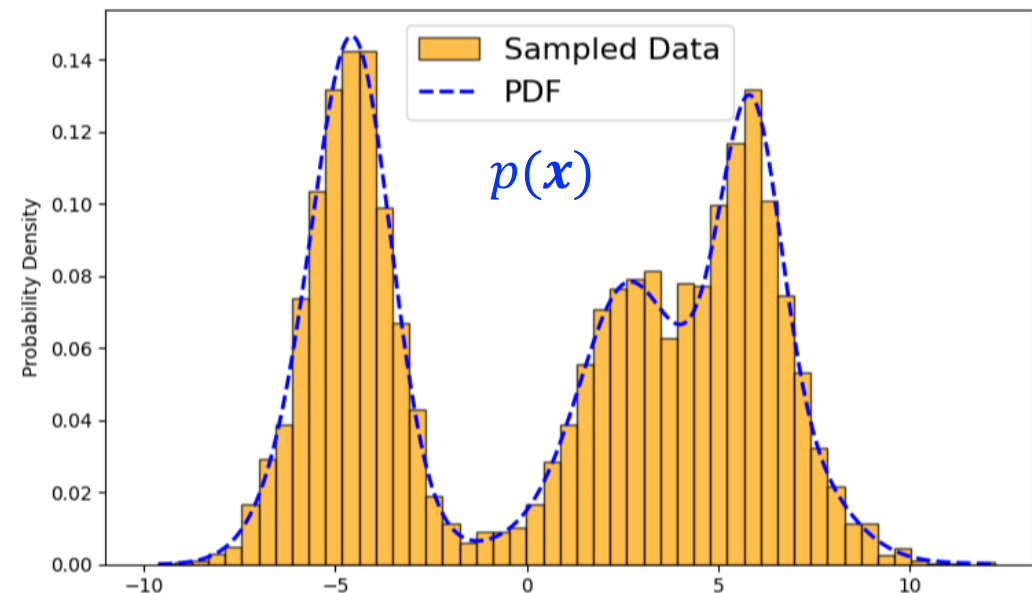
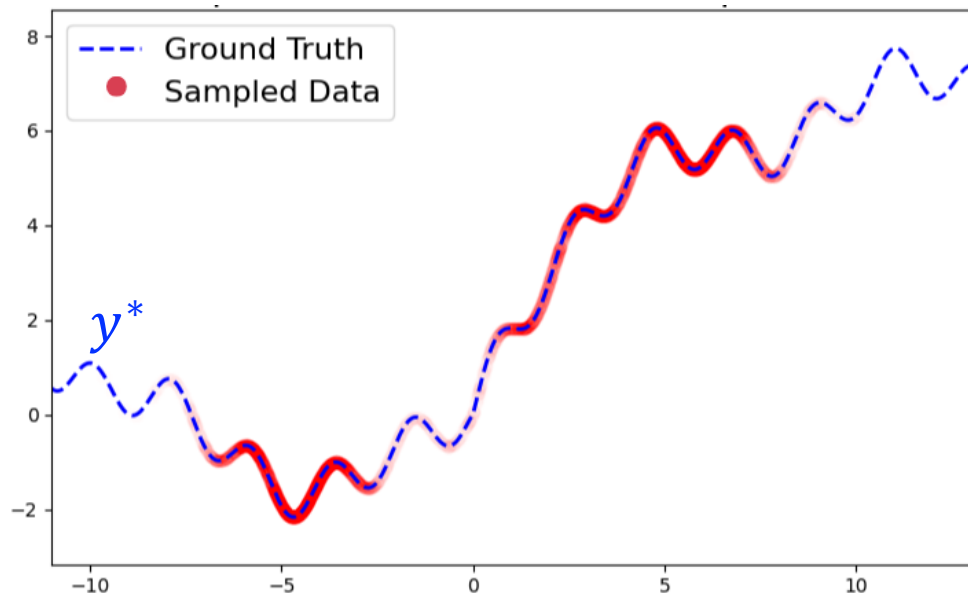
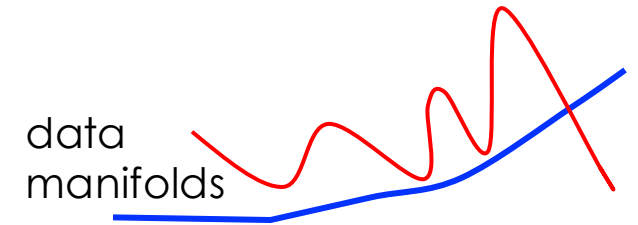
■ Training algorithm $\mathcal{A}$

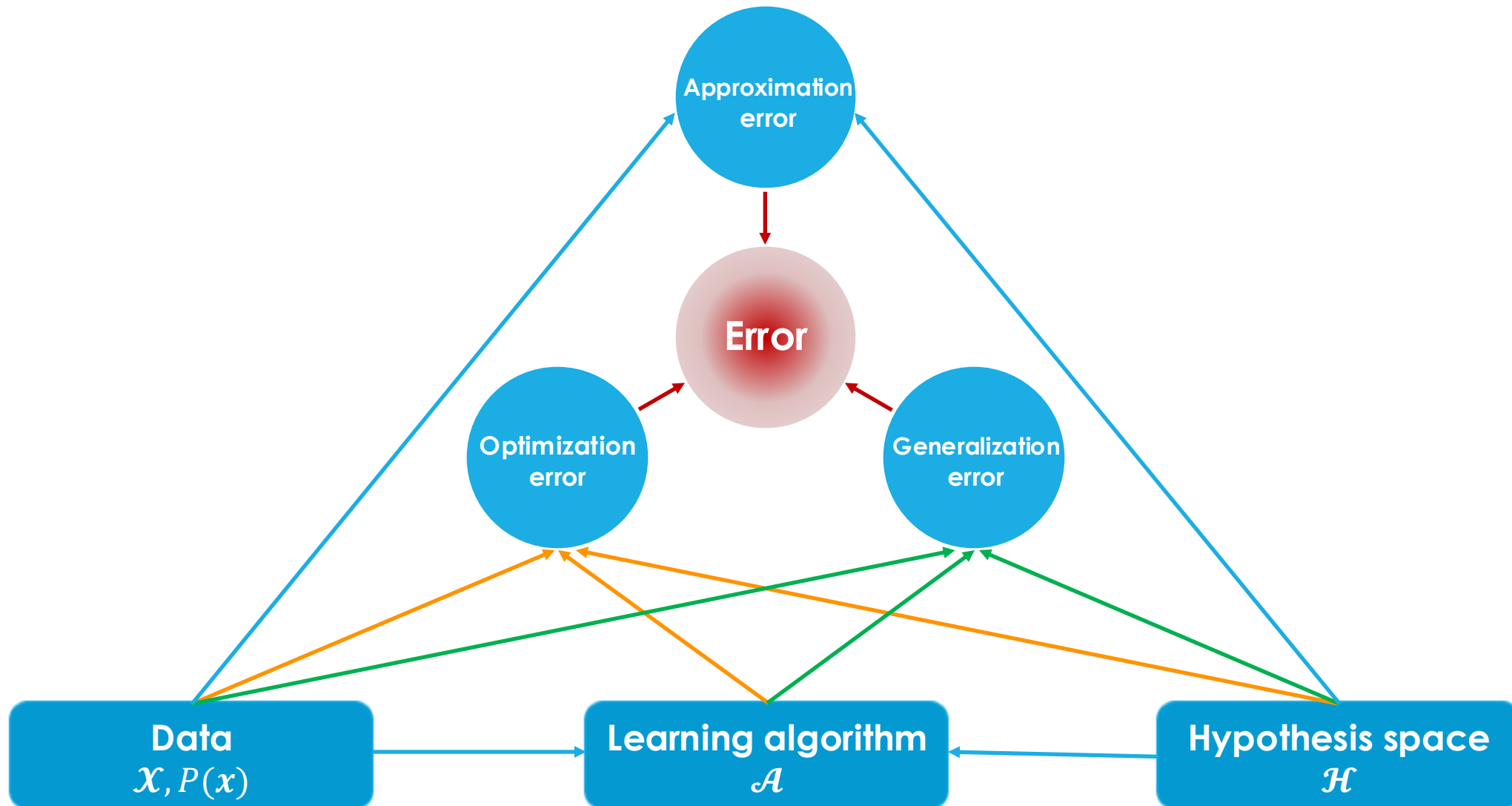
- A better $\mathcal{A}$ implies smaller estimation error
- A **bad** $\mathcal{A}$ can provide small optimization error, but large generalization error → **overfitting**
- A good $\mathcal{A}$ can localize an *effective* subset $\mathcal{H}^* \subset \mathcal{H}$



Errors by different factors (cont.)

- **Data:** manifold c trng không gian d liu
 - Complexity of the data space
- **Data:** distribution
 - Complexity of the sampling distribution
 - Representativeness of the training samples, ...





- Study upper (and lower) bounds for the errors

- **Approximation error:**

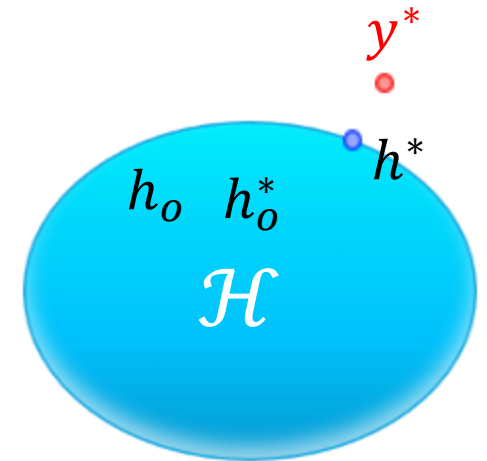
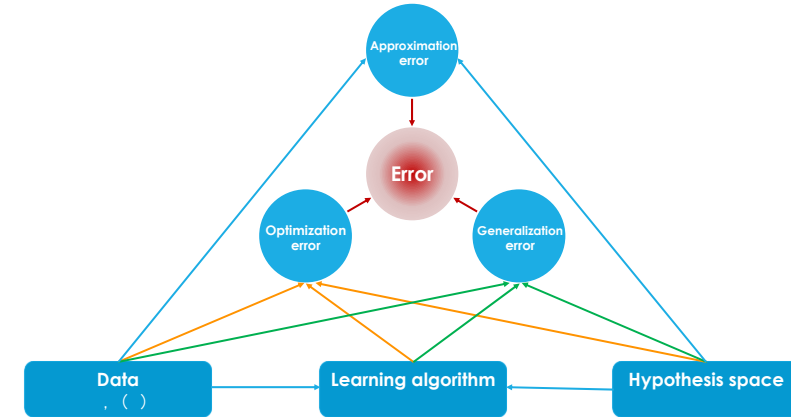
$$|F(P, y^*) - F(P, h^*)| \leq \epsilon_a$$

- Capacity of family $\mathcal{H}$

- *Optimization error:*

$$|F(\mathbf{D}, h_o^*) - F(\mathbf{D}, h_o)| \leq \epsilon_o$$

- Depending on the capacity of learning algorithm $\mathcal{A}$
- and on the used optimizer



$$|F(P, h_o) - F(\mathbf{D}, h_o)| \leq \epsilon_g$$

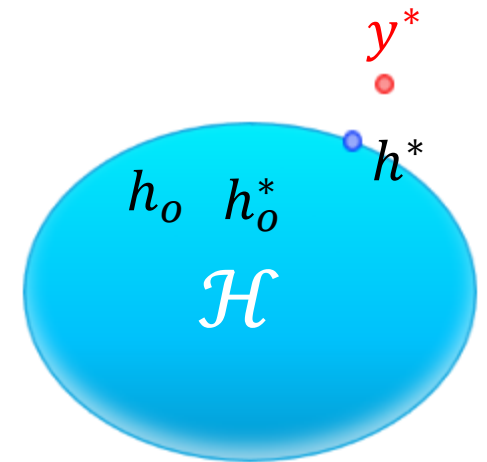
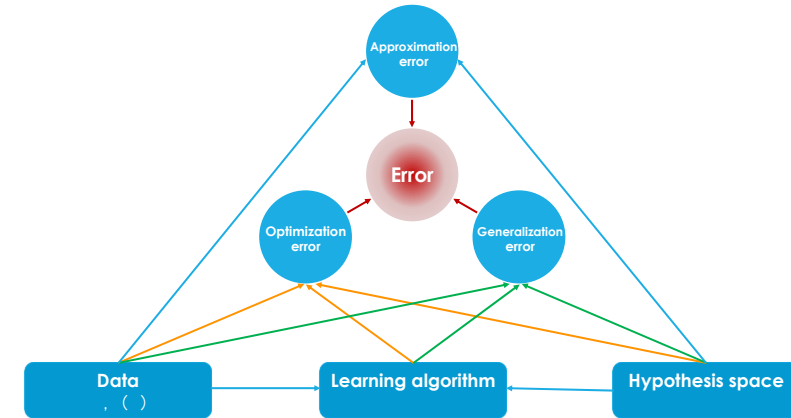
- Generalizability of a learned function h_o
- Uniform bounds:

$$\sup_{h \in \mathcal{H}} |F(P, h) - F(\mathbf{D}, h)| \leq \epsilon_g$$

- Focus on the **worst** member
- May not be a good way to explain a learned function h_o
- PAC-Bayes bounds:

$$|\mathbb{E}_{h \in \mathcal{H}} [F(P, h) - F(\mathbf{D}, h)]| \leq \epsilon_g$$

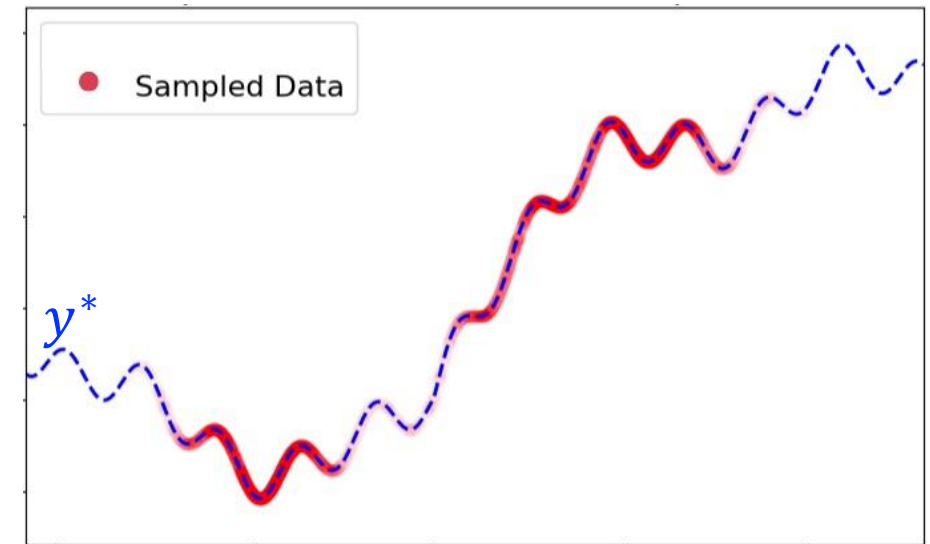
- Study the error on average over $\mathcal{H}$
- May not explain a learned function h_o



Learnability

Generalization ability

- Consider a model $h_o \in \mathcal{H}$
 - Learned from a training set $\mathbf{D} \rightarrow h_o$ depends on $\mathbf{D}$,
 - h_o is a function of $\mathbf{D}$, and we can write $h_o = h_o(\mathbf{D})$
- We want to estimate $F(P, h_o)$
 - How well can h_o generalize on unseen data?
 - But P is unknown !!!
 - Note that $F(P, h_o)$ depends on $\mathbf{D}$
 - When $\mathbf{D}$ changes, so does $F(P, h_o)$
- Training set $\mathbf{D}$
 - Generated by a random process $\rightarrow$ sometimes good, sometimes bad for training
 - There is always some probability that $\mathbf{D}$ happens to be very nonrepresentative of the underlying distribution P



Probabilistic bounds

- An estimate of $F(P, h_o)$ should be in the form of **probabilistic bounds**

$$\Pr(F(P, h_o) > \epsilon) < \delta$$

- Interpret:

- The expected loss is greater than ϵ , with probability at most δ
(xác suất để lỗi trung bình vượt quá ϵ là nhỏ hơn δ)
- δ is the probability of getting a nonrepresentative sample $\mathbf{D}$, so that $F(P, h_o)$ is large
- ϵ is our prediction about the expected loss (phán đoán của ta về lỗi trung bình)

- Hope: *small ϵ and δ*

- Probability to sample a training set for which $F(P, h_o)$ is small

$$\Pr(F(P, h_o) \leq \epsilon) = \Pr(\{\mathbf{D}: F(P, h_o) \leq \epsilon\}) \geq 1 - \delta$$

- $1 - \delta$ represents **confidence** of our prediction ϵ



Approximately correct predictor

- When $F(P, h_o) \leq \epsilon$, we call h_o as *approximately correct predictor*
- The **failure** of the learner: $F(P, h_o) > \epsilon$
- We want to **upper bound** the probability of this event:

$$\Pr(F(P, h_o) > \epsilon)$$

- When $\mathbf{D}$ contains m i.i.d samples from P , we can consider $\mathbf{D}$ to be generated from distribution $P^m = P \times P \times \dots \times P$. Then

$$\Pr(F(P, h_o) > \epsilon) = P^m(\{\mathbf{D}: F(P, h_o) > \epsilon\})$$

- Tỷ lệ của vùng mà chứa các tập mẫu sẽ tạo ra lỗi lớn
- Sometimes we want to **lower bound**

$$\Pr(F(P, h_o) > \epsilon) > ???$$

Provably approximately correct (PAC)

■ **Learnability:** How to define it?

- PAC learning was introduced by Leslie Valiant (1984), the winner of the 2010 Turing Award for the introduction of the PAC model

- **PAC learnability:** A hypothesis class $\mathcal{H}$ is *PAC learnable* if there exist a learning alg. and a function $m_{\mathcal{H}}: (0,1) \times (0,1) \rightarrow \mathbb{N}$ with the following property:
For every $\epsilon, \delta \in (0,1)$, for every distribution P over $\mathcal{X}$, and for every labeling function $y^: \mathcal{X} \rightarrow \mathcal{Y}$, if the realizable assumption holds, then when running the learning algorithm on $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ i.i.d. examples generated by P and labeled by y^* , the algorithm returns a hypothesis h_o such that $F(P, h_o) \leq \epsilon$ with probability of at least $1 - \delta$ (over the choice of the examples)*

$$\Pr(F(P, h_o) \leq \epsilon) \geq 1 - \delta$$

- **Realizable assumption:** $F(P, h^*) = 0$ for the best member $h^* = \arg \min_{h \in \mathcal{H}} F(P, h)$
- The accuracy parameter ϵ determines how far h_o can be from the optimal one (this corresponds to the “approximately correct”),
- The confidence δ indicates how likely h_o is to meet that accuracy requirement (corresponds to the “probably” part of “PAC”).

Agnostic PAC learnability

- Realizable cases are rare in practice
 - They require $\mathcal{H}$ to be too huge to “contain” the truth y^*
- **Non-realizable** cases: $F(P, h^*) \neq 0$ for the best member h^*
 - Hard problems
 - Small family $\mathcal{H}$, or each member is not complex enough
- **Agnostic PAC learnability:** ... when running the learning algorithm on $m \geq m_H(\epsilon, \delta)$ i.i.d. examples generated by P and labeled by y^* , the algorithm returns a hypothesis h_o such that, with probability at least $1 - \delta$ (over the choice of the examples):

$$F(P, h_o) \leq F(P, h^*) + \epsilon$$

- No learner can learn a model with arbitrarily small error
- A learner can still declare success if its error is not much larger than the best error achievable by a predictor from $\mathcal{H}$

Sample complexity

- **Sample complexity:** the minimal number of examples are required to guarantee a probably approximately correct solution
- **Lemma:** if the realizable assumption holds, every finite hypothesis class is PAC learnable with sample complexity $m_H(\epsilon, \delta) \leq \left\lceil \frac{\log(|\mathcal{H}|/\delta)}{\epsilon} \right\rceil$
 - E.g., learning by ERM (see later)
- $m_H(\epsilon, \delta)$ is often required to be *polynomial* in $1/\epsilon$ and $\log(\delta)$
 - to say learnability of $\mathcal{H}$ [Valiant, 1984]
- **The curse of dimensionality:** $m_H(\epsilon, \delta)$ is **exponential** in the dimensionality n
- **Exercise:** Show that $m_H(\epsilon, \delta)$ is monotonically nonincreasing in each of its parameters, when $\mathcal{H}$ is PAC learnable

No-free lunch theorem

- For one problem, there are many learners
 - Some are really powerful, e.g., using SGD
 - More training samples can help training loss closer and closer to expected loss
 - ➔ Meaning the trained model can be better
- **Is there a universal learner that can learn any functions** just by increasing training size?
- **No-free lunch theorem:** *Let $\mathcal{A}$ be any learner for the task of binary classification with input space $\mathcal{X}$, the 0-1 loss, and any training size $m \leq |\mathcal{X}|/2$. Then there exists a distribution P over $\mathcal{X} \times \{0, 1\}$ satisfying:*
 - *There exists a function h with $F(P, h) = 0$*
 - *$F(P, \mathcal{A}(\mathbf{D})) \geq 1/8$, with probability of at least $1/7$ over the choice of $\mathbf{D} \sim P^m$*
- **No universal learner !**